10PEARLS DATA SCIENCE INTERNSHIP

PROJECT REPORT

AQI Next 3 Days Predictor

Name	Ahmed Mallick
Organization	10Pearls
Project	AQI Next 3 Days Predictor
Target City	Karachi, Pakistan

Streamlit App

1. Executive Summary & Serverless Architecture

The AQI Next 3 Days Predictor is an end-to-end, serverless machine learning system developed to forecast air-quality conditions in Karachi for the subsequent three days. The project integrates automated data ingestion, feature engineering, cloud-based feature management, model training, model registration, continuous automation, and an interactive web interface into a single production-oriented workflow. Rather than treating model training as an isolated notebook exercise, the solution is structured as a repeatable pipeline capable of receiving new observations, updating its feature store, retraining models, and exposing forecasts through a web application.

The architecture separates the system into clear layers. OpenWeather acts as the external source for weather and air-pollution observations. Hopsworks provides the cloud feature-store and model-management layer, allowing historical and newly ingested features to be persisted and reused consistently. GitHub Actions provides the automation layer for scheduled ingestion and model-training workflows. Finally, Streamlit Community Cloud provides the presentation layer through which users can inspect recent conditions and the 72-hour forecast.

At a high level, the data flow is: OpenWeather API → ingestion and feature engineering → Hopsworks Feature Store → scheduled model training and Model Registry → Streamlit inference application. This serverless design minimizes the need for continuously running infrastructure while retaining the operational characteristics expected from an enterprise machine-learning workflow.

1.1 Architecture Overview

Layer	Technology	Primary Responsibility	Output
Data Source	OpenWeather APIs	Weather and pollution observations	Raw observations
Data Engineering	Python pipelines	Cleaning, time features, derived variables	Model-ready features
Feature & Model Platform	Hopsworks	Feature Store and Model Registry	Versioned features/models
Automation	GitHub Actions	Scheduled ingestion and retraining	Updated pipeline state
Application	Streamlit Community Cloud	Forecast visualization and interpretation	72-hour forecast dashboard

1.2 System Objectives

- Forecast Karachi's AQI for the next three days using recent air-quality and weather information.
- Establish a reproducible data pipeline rather than relying on manually prepared datasets.
- Persist historical and live features in a centralized feature store for consistency between training and inference.
- Automate feature ingestion and model refresh through scheduled CI/CD workflows.
- Provide an accessible dashboard with forecasts, exploratory analytics, feature explanations, and safety warnings.
- Demonstrate practical MLOps capabilities including model benchmarking, registry management, deployment, and dependency control.

2. Data Extraction & Feature Store Pipeline

The data-engineering layer establishes the foundation of the forecasting system. The initial historical dataset is generated through the backfill process, represented by the project's backfill script (backfill pipeline). The process uses OpenWeather and Hopsworks APIs to obtain approximately six months of historical observations and populate the feature-store workflow. Historical backfilling is important because a forecasting model requires a sufficiently large temporal context before it can be evaluated and deployed against live data.

After retrieval, the pipeline transforms raw observations into structured analytical features. Time-based variables such as hour, day, and month are extracted from timestamps so that the model can learn recurring temporal patterns. In addition, domain-oriented derived variables are generated, including an AQI change-rate signal that captures the direction and relative magnitude of recent air-quality movement. These transformations convert raw API responses into features suitable for supervised learning.

2.1 Historical Backfill

1. Connect to OpenWeather using the configured API credentials and request historical weather and pollution observations for the target location.
2. Normalize the returned records into a consistent tabular structure with timestamps and relevant weather/AQI variables.
3. Compute time-derived fields such as hour, day, and month.
4. Calculate derived signals, including the AQI change rate, where sufficient preceding observations are available.
5. Write the transformed records into the Hopsworks Feature Store so that historical training data and subsequent live observations share a common representation.

2.2 Live Feature Ingestion

Following the historical backfill, the same feature-engineering logic is applied to newly available observations. The live pipeline periodically fetches the latest data, performs the required transformations, and records the resulting features in Hopsworks. Keeping historical and live features in the same feature-store workflow reduces training-serving skew and creates a consistent source of truth for downstream model operations.

2.3 Feature Engineering

Feature Group	Examples	Purpose
Temporal	hour, day, month	Capture daily and seasonal patterns
Air Quality	AQI and recent AQI movement	Represent current pollution state and trend
Weather	Available OpenWeather meteorological variables	Capture environmental conditions affecting air quality
Derived	AQI change rate and related transformations	Provide trend-sensitive signals for forecasting

3. ML Training & CI/CD Automation

The machine-learning layer converts the engineered feature set into a forecasting model and operationalizes model updates. Multiple candidate algorithms are benchmarked rather than assuming that a single model family will perform best. The project evaluates approaches including Random Forest, Ridge Regression, and TensorFlow-based modeling. Model performance is assessed using RMSE, MAE, and R-squared, providing complementary views of prediction error and explained variance.

The selected model is committed to the Hopsworks Model Registry, establishing a managed artifact that can be retrieved by the inference application. This registry-oriented workflow separates experimentation from deployment and provides a clearer path toward reproducible model promotion.

3.1 CI/CD Workflow Design

Workflow	Frequency	Core Actions	Operational Goal
Feature Pipeline	Hourly	Fetch new OpenWeather observations, transform features, write to Hopsworks	Keep feature data current
Training Pipeline	Daily	Load features, train/benchmark models, evaluate metrics, register updated model	Keep model current

3.2 Evaluation Metrics

- RMSE (Root Mean Squared Error): emphasizes larger prediction errors and is useful for identifying significant forecast deviations.
- MAE (Mean Absolute Error): provides an interpretable average magnitude of prediction error.
- R^2 (R-squared): measures the proportion of variance explained by the model relative to a baseline.

Using multiple metrics prevents model selection from depending on a single statistical view. The benchmark results can therefore be considered collectively when deciding which model is appropriate for deployment.

4. Dashboard Deployment & Advanced Analytics

The prediction service is exposed through a Streamlit web application deployed on Streamlit Community Cloud. The application is designed as the user-facing layer of the pipeline: it retrieves the latest feature context and/or registered model outputs, generates a 72-hour forecast, and presents the results in an accessible dashboard. This makes the machine-learning system usable by stakeholders who do not need direct access to notebooks, APIs, or cloud infrastructure.

The dashboard goes beyond displaying a single prediction. It includes exploratory data analysis visualizations to communicate historical patterns and current conditions, while SHAP integration supports explainability by showing which input variables contribute most strongly to model predictions. This is particularly valuable for an environmental forecasting application because users can inspect not only the forecast but also the factors influencing it.

A safety-oriented presentation layer is also included. When the OpenWeather AQI index reaches severe categories (index 4 or 5), the application presents prominent warning banners. The intent is to distinguish ordinary analytical output from conditions that may require immediate user attention.

4.1 Dashboard Capabilities

- 72-hour / three-day AQI forecast visualization.
- Current and recent air-quality context.
- Exploratory Data Analysis charts for understanding distributions and temporal behavior.
- SHAP-based feature-importance explanations to improve model interpretability.
- Prominent severe-condition warnings for OpenWeather AQI index levels 4 and 5.

- Cloud-hosted access through Streamlit Community Cloud without requiring users to run the application locally.

5. Challenges & Learnings

A major practical challenge arose during Streamlit deployment from dependency conflicts in requirements.txt. Machine-learning projects often combine libraries with different release cycles and transitive dependencies. A package set that works in a development environment may fail during cloud deployment because the deployment environment resolves versions differently or because one library requires an incompatible version of another dependency.

Resolving these conflicts required reviewing the project's package requirements, identifying incompatible or unnecessary dependencies, and aligning versions so that the application could install reliably in the target environment. This experience highlighted an important distinction between model development and production deployment: successful experimentation is not sufficient unless the complete runtime environment is reproducible.

- Dependency management is a core MLOps responsibility, not merely an installation detail.
- Automated pipelines must handle credentials, environment variables, external API availability, and package compatibility consistently.
- Feature stores reduce the operational burden of maintaining separate training and inference data preparation logic.
- Scheduled retraining creates a repeatable mechanism for keeping a forecasting model aligned with changing data.
- Explainability and safety messaging are important when presenting machine-learning predictions to non-technical users.
- Cloud deployment exposes integration issues that may remain invisible during local development, making deployment validation essential.

6. End-to-End Operational Workflow

1. Historical backfill establishes the initial six-month feature history using OpenWeather data.
2. The feature engineering process creates temporal and domain-derived variables.
3. Historical and live features are stored in Hopsworks Feature Store.
4. The hourly GitHub Actions workflow ingests new observations and updates the feature store.
5. The daily training workflow retrieves the latest training data and benchmarks candidate models.
6. Performance is evaluated using RMSE, MAE, and R^2.
7. The selected model is registered in Hopsworks Model Registry.
8. The Streamlit application uses the deployed model to produce a three-day / 72-hour forecast.
9. The dashboard communicates forecast values, EDA insights, SHAP explanations, and severe AQI warnings.

7. Project Outcomes

The completed solution demonstrates an integrated data-science lifecycle rather than a standalone predictive model. It combines data acquisition, feature engineering, cloud feature management, model experimentation, automated retraining, model registration, explainable analytics, and cloud deployment. From an internship

perspective, the project provides practical evidence of applying Data Science and MLOps principles to a real-world environmental forecasting problem.

Capability	Demonstrated Implementation
Data Engineering	API-based extraction, historical backfill, transformation and derived features
Feature Management	Hopsworks Feature Store for historical and live features
Machine Learning	Benchmarking of Random Forest, Ridge Regression and TensorFlow approaches
Model Evaluation	RMSE, MAE and R^2
MLOps / CI-CD	Hourly feature ingestion and daily training automation with GitHub Actions
Model Management	Hopsworks Model Registry
Explainable AI	SHAP feature-importance analysis
Deployment	Streamlit Community Cloud
User Safety	Severe AQI warnings for OpenWeather index 4 and 5

8. Conclusion

The AQI Next 3 Days Predictor represents a production-oriented implementation of an end-to-end machine-learning pipeline for Karachi air-quality forecasting. Its architecture demonstrates how external environmental data can be transformed into managed features, used for continuously refreshed models, and delivered through a serverless application. The integration of scheduled automation, model registry practices, explainability, and deployment-focused dependency management strengthens the project beyond a conventional academic ML exercise.

The project also demonstrates an important professional lesson: reliable Data Science requires coordination across the entire lifecycle from data acquisition and feature quality to model evaluation, automation, deployment, and user communication. The resulting system provides a foundation that can be extended with richer historical data, additional meteorological variables, more advanced time-series architectures, monitoring, drift detection, and formal model-promotion policies.

Technology Stack

Category	Technology	Role
Data Source	OpenWeather	Weather and air-pollution data acquisition
Programming	Python	Data processing, feature engineering

		and ML pipelines
Feature Store	Hopsworks	Centralized feature storage and retrieval
Model Registry	Hopsworks Model Registry	Model artifact management
Automation	GitHub Actions	Scheduled CI/CD workflows
Dashboard	Streamlit	Interactive forecasting application
Cloud Deployment	Streamlit Community Cloud	Application hosting
Explainability	SHAP	Feature contribution analysis
ML Libraries	Scikit-learn / TensorFlow	Model development and benchmarking